Viriato

Relatório do Sistema

Controle de processos multi-empresa: como o sistema funciona, como foi construído e quais são seus limites atuais.

Versão do código	commit `6b83214` (branch `claude/awesome-hopper-3on31d`)
Data	10 de novembro de 2026
Estágio	MVP funcional, ainda não preparado para produção
Repositório	github.com/monfredo18/Viriato

1. Visão geral

O Viriato é um sistema web para cadastrar, acompanhar e automatizar **processos** dentro de uma empresa. Cada processo percorre as **etapas** de um **fluxo** definido pela própria empresa, tem um responsável, um prazo por etapa, documentos anexados e um histórico completo do que aconteceu.

O sistema foi desenhado desde o início como **SaaS multi-empresa**: uma única instalação atende várias empresas, e os dados de uma nunca são visíveis para outra.

> **Em uma frase:** a empresa define um fluxo (por exemplo Abertura → Análise → Revisão → Concluído), abre processos nesse fluxo, e o sistema controla quem é o responsável, em que etapa cada processo está, quando vence o prazo e o que já foi feito.

O que o sistema faz hoje

- **Cadastro self-service de empresa:** quem se cadastra vira administrador e já recebe um fluxo padrão.
- **Usuários e papéis:** Administrador, Gestor e Usuário, com permissões diferentes.
- **Fluxos configuráveis:** etapas com prazo em dias, etapa inicial e etapas finais.
- **Processos:** criação, busca e filtros, movimentação entre etapas respeitando as regras do fluxo, atribuição, comentários e cancelamento.
- **Histórico:** cada ação fica registrada com autor, data e comentário.
- **Documentos:** upload e download de arquivos por processo, guardados no AWS S3.
- **Prazos e notificações:** aviso dentro do sistema (sininho) e por e-mail, com alerta automático de prazo vencendo ou vencido.
- **Interface:** tema claro e escuro com botão de lua e sol.

Glossário

TERMO	SIGNIFICADO
Empresa (tenant)	Cliente do sistema. Todos os dados pertencem a uma empresa.
Fluxo	Modelo de trabalho: conjunto de etapas e das transições permitidas entre elas.
Etapa	Um passo do fluxo. Pode ter prazo em dias, ser a inicial ou ser final.
Transição	Caminho permitido de uma etapa para outra. Só movimentos previstos são aceitos.
Processo	Um caso concreto em andamento num fluxo (contrato, alvará, licença etc.).
Responsável	Usuário encarregado do processo no momento. Recebe os avisos.

2. Arquitetura e tecnologias

O sistema é um **monolito modular**: um único backend dividido em módulos por assunto, o que é mais simples de operar do que microsserviços e permite separar partes depois, se for preciso.

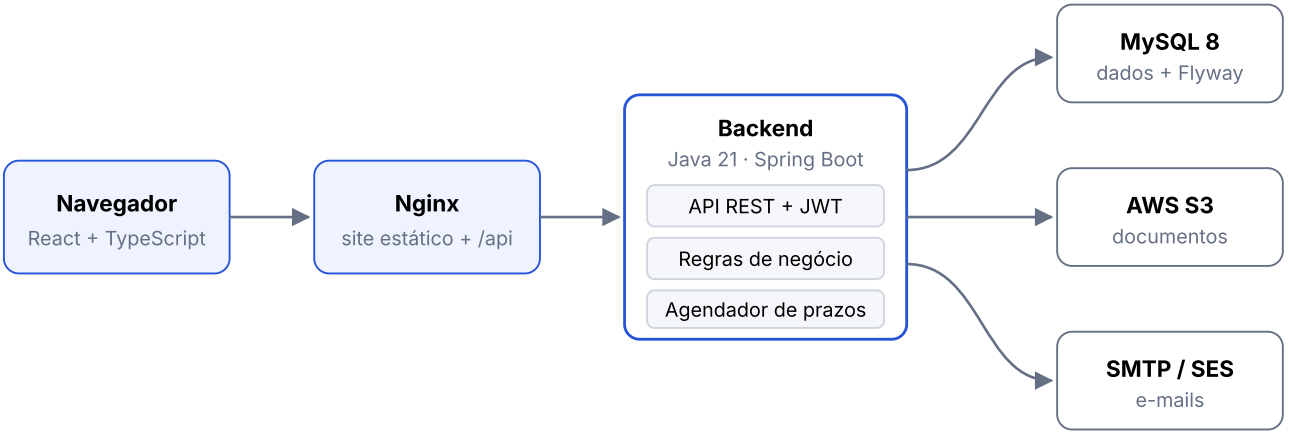


Figura 1 — Visão geral. O navegador só fala com o Nginx; o Nginx entrega o site e repassa `/api` ao backend.

Tecnologias

CAMADA	TECNOLOGIA	PAPEL
Interface	React 18, TypeScript, Vite, React Router	Telas, rotas, tema claro/escuro
Backend	Java 21, Spring Boot 3.3 (Web, Data JPA, Security, Validation, Mail)	API REST, regras, segurança
Banco	MySQL 8.4, Flyway	Dados e versionamento do esquema
Autenticação	JWT (jjwt), BCrypt	Login sem sessão no servidor
Arquivos	AWS S3 (SDK v2)	Documentos anexados
E-mail	Spring Mail (SMTP; AWS SES em produção)	Notificações por e-mail
Documentação da API	springdoc OpenAPI / Swagger UI	Referência interativa em `/swagger-ui.html`
Infraestrutura	Docker, Docker Compose, Nginx	Execução local e base para produção
Qualidade	JUnit 5, MockMvc, H2, GitHub Actions	Testes automáticos e CI

Organização do código

MÓDULO (BACKEND)	RESPONSABILIDADE
`seguranca`	Geração e validação do JWT, filtro de requisições, regras de acesso e CORS.
`usuario`	Login, cadastro de empresa (onboarding), usuários e papéis.
`fluxo`	Fluxos, etapas e transições por empresa.
`processo`	Núcleo do sistema: processos, movimentação, histórico, atribuição, cancelamento.
`documento`	Upload, download e remoção de arquivos, com armazenamento abstraído (S3).
`notificacao`	Avisos no sistema, envio de e-mail e agendador de prazos.
`comum`	Tratamento padronizado de erros e utilitários de data.

3. Como o sistema funciona

Esta seção percorre a jornada de uso, da primeira tela até o encerramento de um processo.

3.1 Primeiro acesso: cadastro da empresa

Na tela de login, *Cadastrar minha empresa* pede o nome da empresa, o nome do usuário, e-mail e senha (mínimo de 8 caracteres). Numa única operação o sistema cria a empresa, o primeiro usuário como **Administrador** e um **fluxo padrão**, e já devolve a sessão. Não existe usuário ou senha pré-cadastrados.

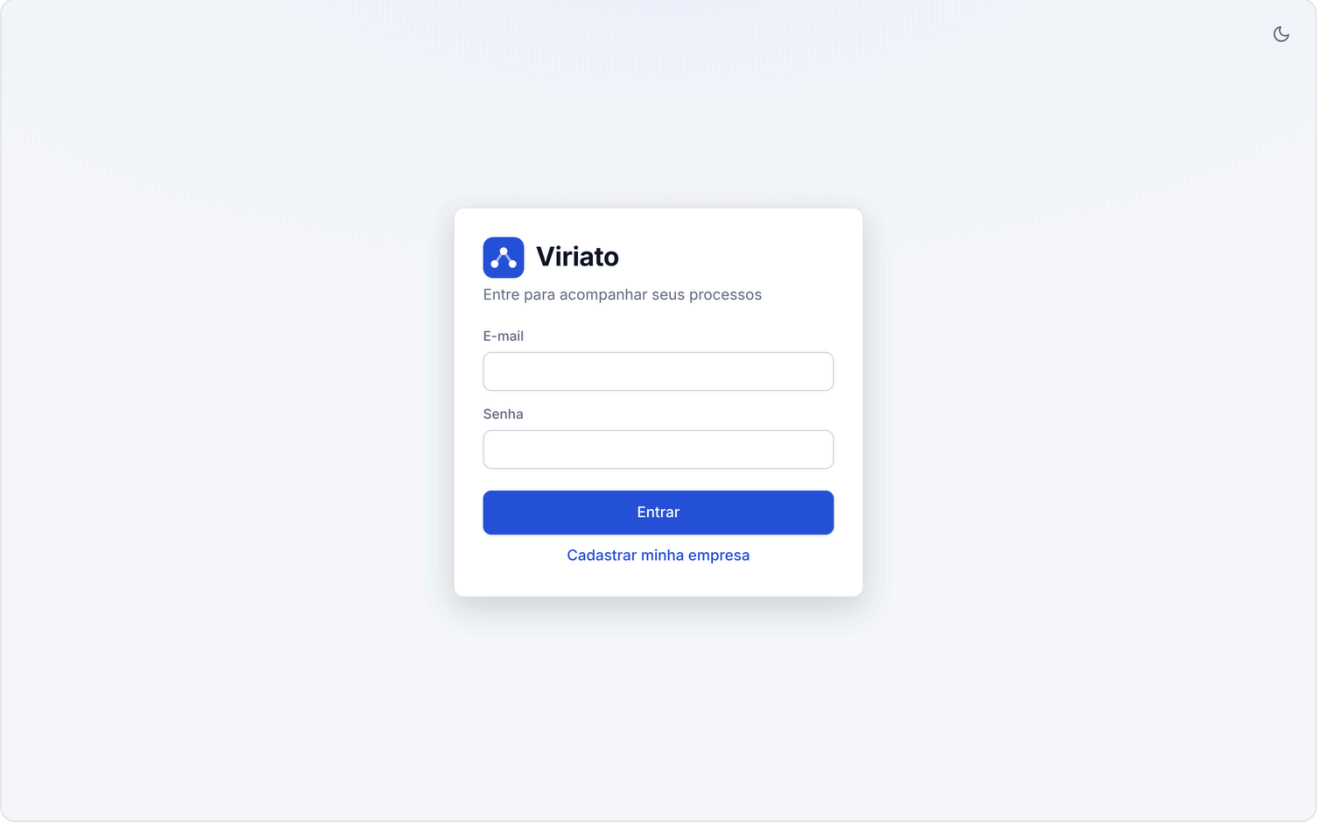


Figura 2 — Tela de login, com o botão de tema no canto superior direito.

3.2 Login e sessão

1. O usuário informa e-mail e senha. O sistema confere a senha (BCrypt) e verifica se o usuário e a empresa estão ativos.
2. Se estiver tudo certo, devolve um **token JWT** válido por 12 horas. Ele carrega o identificador do usuário, o da empresa e o papel.
3. O navegador guarda o token e o envia em cada chamada. Se o servidor responder 401, o usuário é deslogado.

A empresa de cada requisição vem **sempre do token**, nunca de algo que o usuário digite. É isso que sustenta o isolamento entre empresas.

3.3 Fluxos

Administradores e Gestores criam fluxos na tela *Fluxos*: informam as etapas em ordem, o prazo em dias de cada uma, qual é a inicial (exatamente uma) e quais são finais (ao menos uma). Regras validadas pelo sistema:

- mínimo de 2 etapas; todas com nome; prazo não negativo;
- uma etapa final não pode ter saída;
- pela tela, as etapas são ligadas em sequência (cada uma leva à seguinte). A API já aceita transições arbitrárias.

Um fluxo pode ser desativado: deixa de aceitar novos processos, mas os que já existem continuam normalmente.

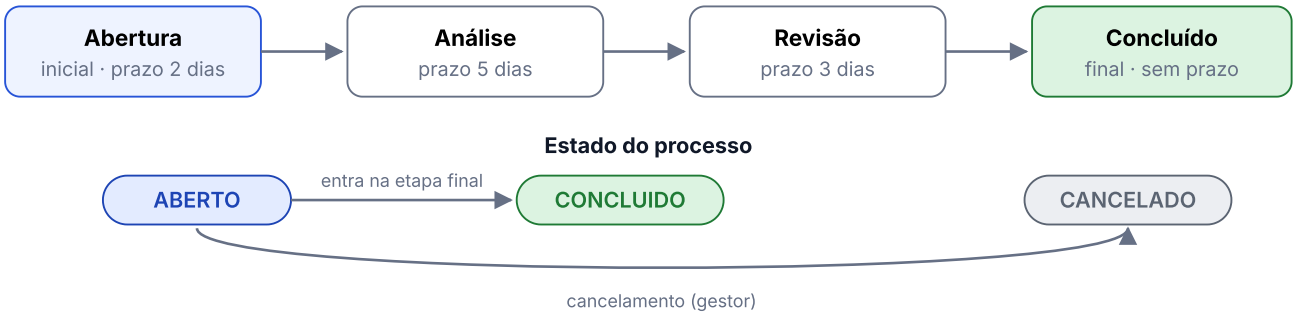


Figura 3 — Fluxo padrão criado com a empresa e os três estados de um processo.

3.4 Abrir e acompanhar processos

Qualquer usuário pode abrir um processo informando título, descrição opcional, fluxo e responsável opcional. Ao criar, o sistema:

1. confere que o fluxo é da empresa e está ativo, e que o responsável é um usuário ativo da empresa;
2. coloca o processo na etapa inicial e calcula o prazo (hoje + dias da etapa, fuso America/Sao_Paulo);
3. gera o número no formato `ANO-NNNNNN` (por exemplo `2026-000001`);
4. registra no histórico a ação *Criado* e avisa o responsável, se for outra pessoa.

A lista de processos tem busca por título ou número, filtro por situação, *Meus* (onde sou responsável) e *Atrasados*, com paginação. Um processo é considerado **atrasado** quando está aberto e o prazo da etapa já passou; isso é calculado na hora da consulta.

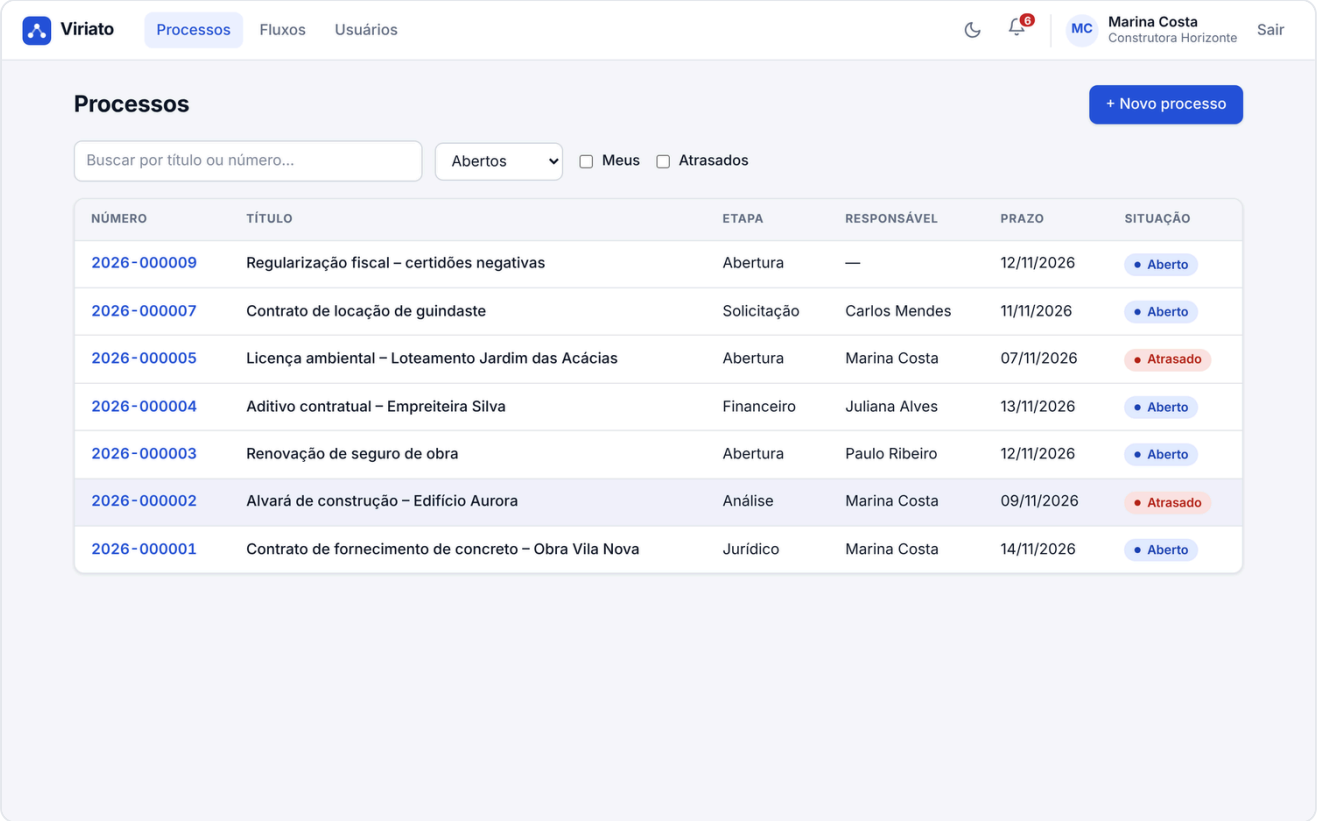


Figura 4 — Lista de processos com busca, filtros e selos de situação (tema claro).

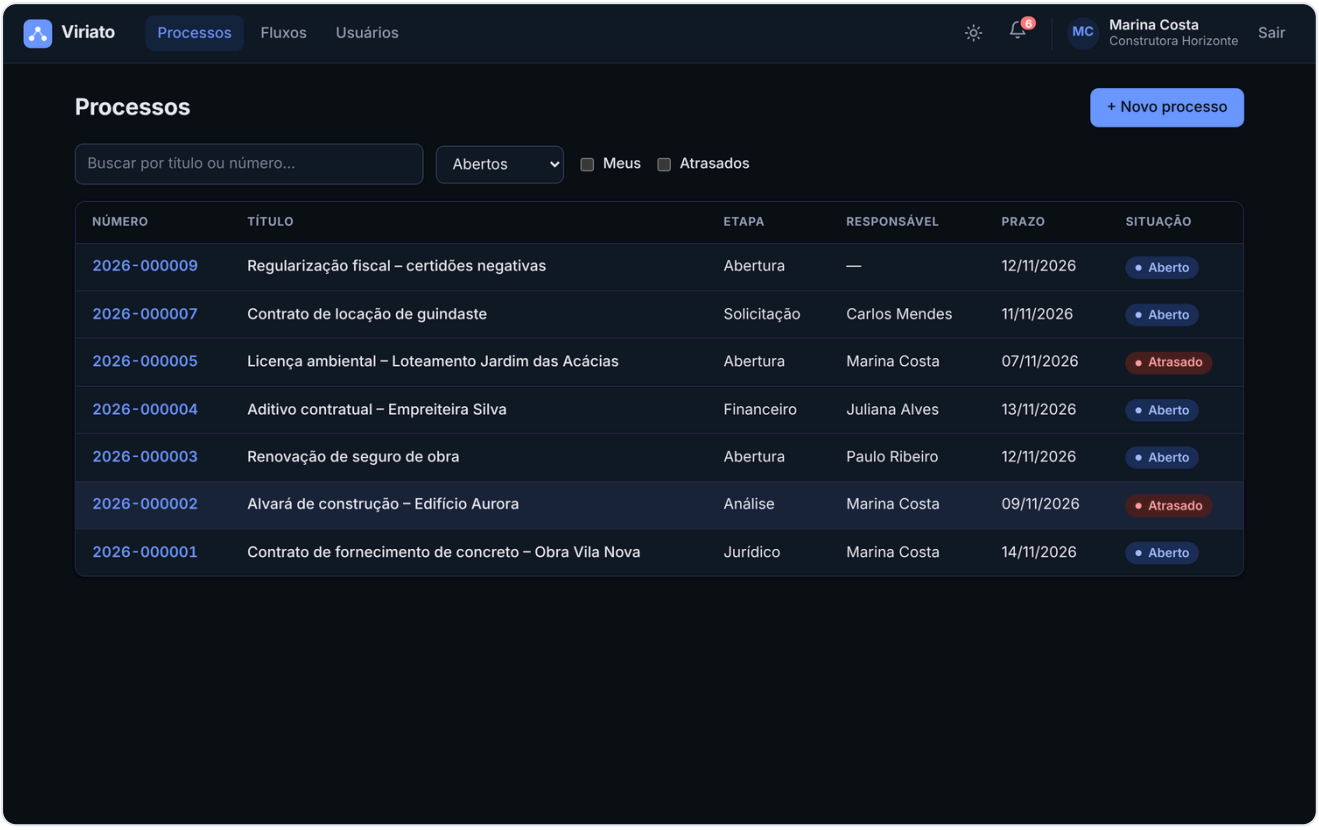


Figura 5 — A mesma tela no tema escuro, escolhido pelo botão de sol/lua no topo.

3.5 Movimentar um processo

No detalhe do processo aparecem os botões das etapas para as quais o fluxo permite avançar a partir da etapa atual. Ao avançar:

1. o processo precisa estar aberto e o usuário precisa ter permissão (ver seção 4);
2. a transição precisa existir no fluxo; senão o sistema recusa com a mensagem *Transição não permitida pelo fluxo*;
3. a etapa muda, o prazo é recalculado e o controle de alerta é zerado;
4. se a etapa de destino for final, o processo vira **Concluído** e o prazo é removido;
5. a movimentação entra no histórico, com o comentário opcional, e o responsável é avisado.

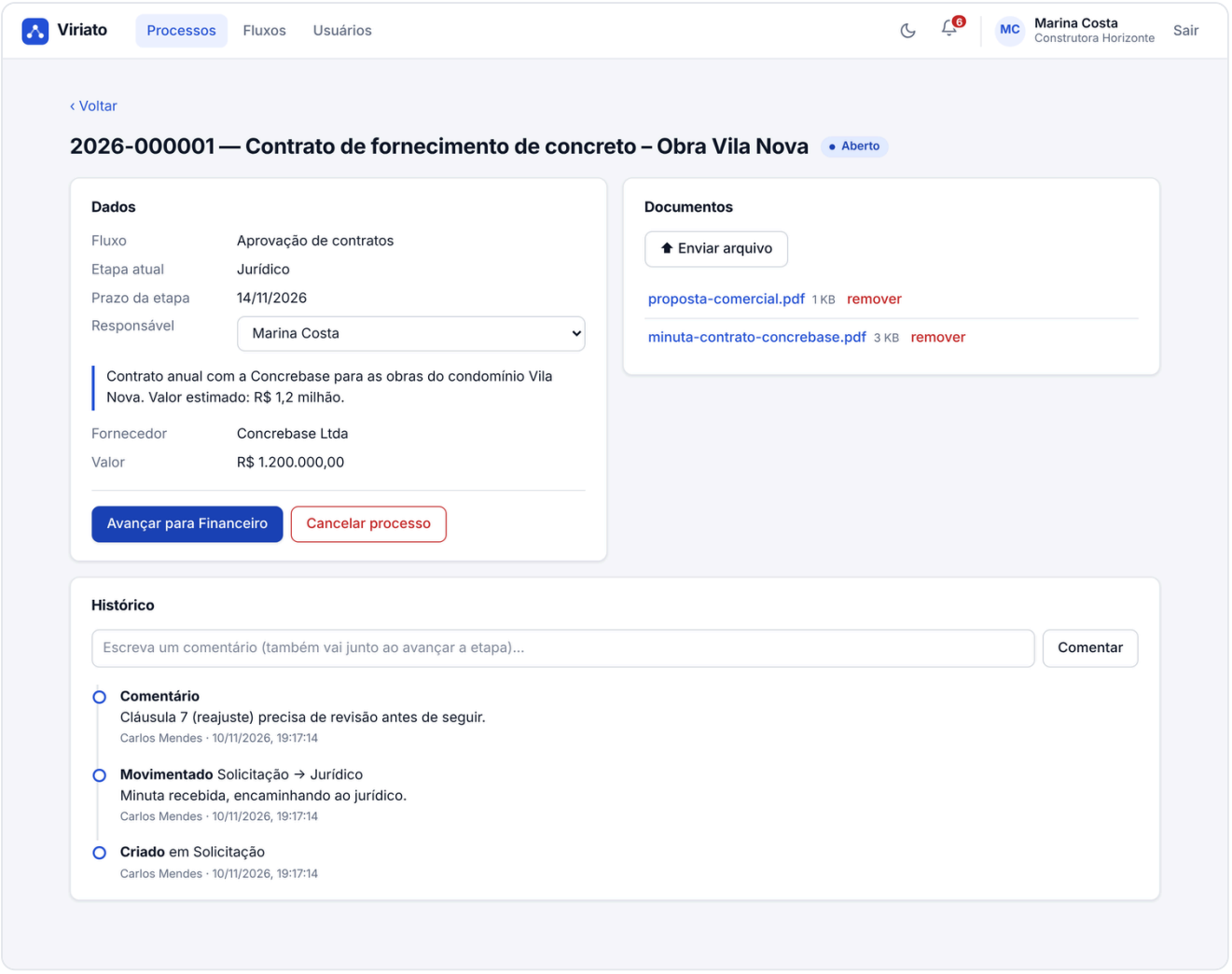


Figura 6 — Detalhe: dados, campos extras, documentos, ações disponíveis e histórico em linha do tempo.

3.6 Documentos

Cada processo aceita anexos de até 25 MB por arquivo. O sistema sanitiza o nome do arquivo (remove caminhos e caracteres perigosos), grava no S3 com uma chave única por empresa e processo e criptografia no repouso (AES-256), e só então registra no banco. Se o registro falhar, o arquivo enviado é removido para não sobrar lixo no bucket. O **download passa pelo backend**, que confere a empresa antes de entregar o arquivo; não há links públicos.

3.7 Prazos e notificações

Uma tarefa agendada roda todos os dias às 08:00 (America/Sao_Paulo). Ela procura processos abertos, com responsável, cujo prazo vence em até 1 dia ou já venceu e que ainda não foram alertados naquela etapa. Para cada um, avisa o responsável (*Prazo próximo do vencimento* ou *Prazo vencido*) e marca como alertado. O resultado é **um alerta por etapa**; ele volta a valer quando o processo muda de etapa ou de responsável.

Cada notificação é gravada no sistema (sininho no topo, com contador de não lidas, atualizado a cada 30 segundos) e enviada por e-mail de forma assíncrona. O e-mail é *best-effort*: se o servidor de e-mail falhar, isso é registrado em log e a operação principal não é afetada.

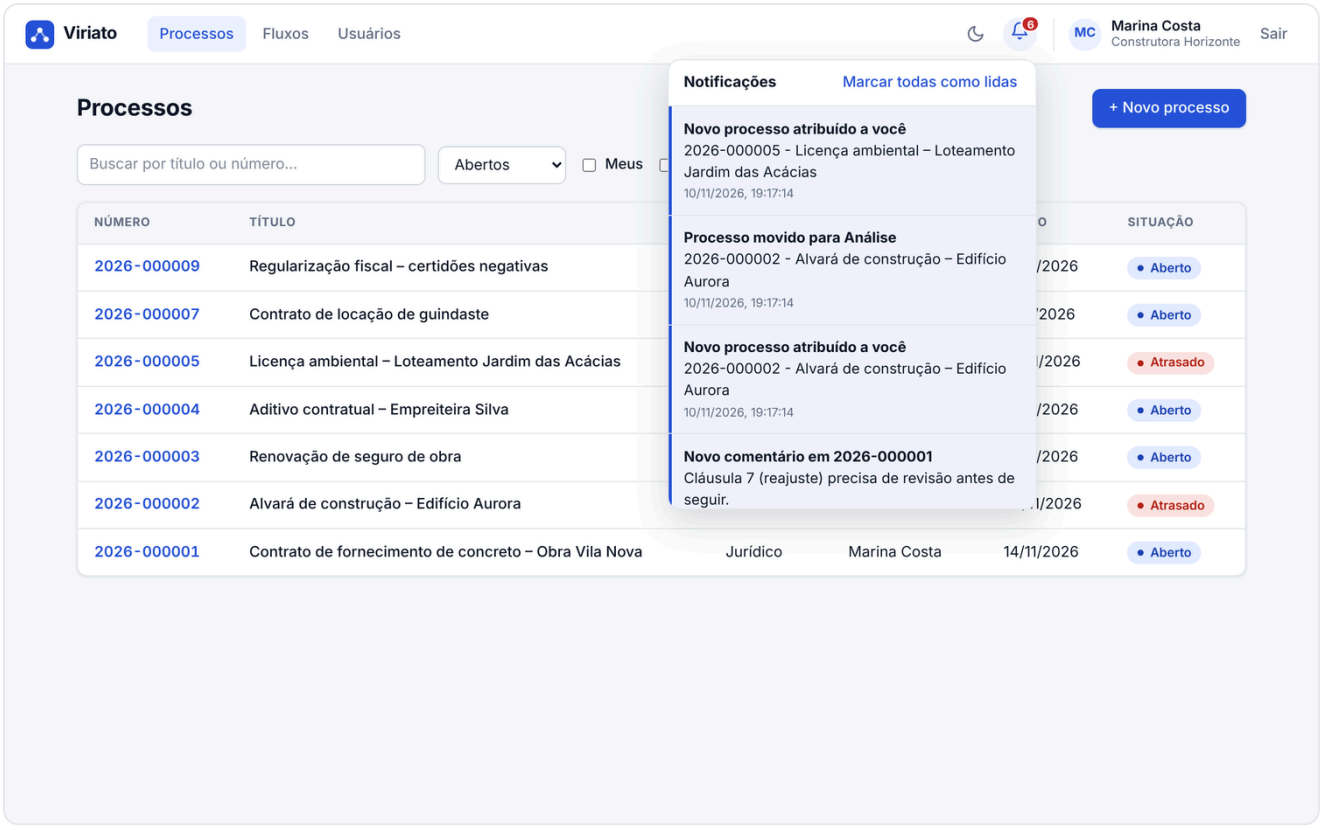


Figura 7 — Painel de notificações; as não lidas têm destaque e levam direto ao processo.

4. Papéis e permissões

AÇÃO	ADMINISTRADOR	GESTOR	USUÁRIO
Ver processos, documentos e histórico da empresa	**sim**	**sim**	**sim**
Criar processo, comentar, anexar documento	**sim**	**sim**	**sim**
Avançar etapa	**sim**	**sim**	só se for o responsável, ou se não houver responsável
Reatribuir responsável	**sim**	**sim**	só se for o responsável atual
Remover documento	**sim**	**sim**	só os que ele mesmo enviou
Cancelar processo	**sim**	**sim**	**não**
Criar e ativar/desativar fluxos	**sim**	**sim**	**não**
Criar usuários, mudar papel, ativar/desativar	**sim**	**não**	**não**

Detalhes de comportamento:

- Recusas por papel (criar fluxo, criar usuário) devolvem HTTP 403. Recusas por regra de negócio (avançar sem ser o responsável, cancelar como Usuário) devolvem HTTP 422 com uma mensagem em português.
- O administrador não consegue desativar nem rebaixar a si mesmo, para evitar ficar sem acesso.
- Recursos de outra empresa respondem 404, como se não existissem.

5. Modelo de dados

Todas as tabelas de negócio têm a coluna `tenant_id`, e toda consulta filtra por ela. O esquema é criado e versionado pelo Flyway (`V1__esquema_inicial.sql`).

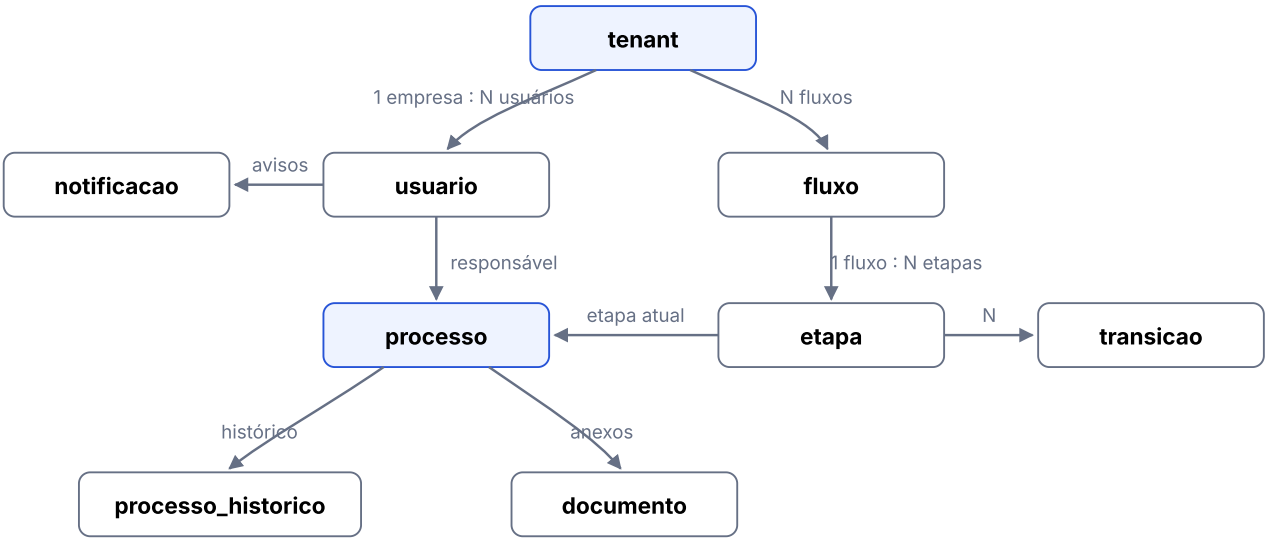


Figura 8 — Relações principais. As setas vão de quem é referenciado para quem o referencia. Por clareza, nem todas as ligações foram desenhadas: todas as tabelas de negócio apontam também para a empresa (`tenant`), e o processo aponta para o fluxo.

TABELA	CONTEÚDO E PONTOS IMPORTANTES
`tenant`	Empresa cliente; flag `ativo`.
`usuario`	Nome, e-mail (único em todo o sistema), hash BCrypt da senha, papel, ativo.
`fluxo`	Nome e flag `ativo`.
`etapa`	Nome, ordem, `prazo_dias` (opcional), flags `inicial` e `final_`.
`transicao`	Par de etapas (de, para), único; define os movimentos permitidos.
`processo`	Número (único por empresa), título, descrição, fluxo, etapa atual, responsável, status, `prazo_etapa`, `prazo_alertado`, `campos_extras` (JSON livre em texto).
`processo_historico`	Ação (CRIADO, MOVIDO, COMENTARIO, ATRIBUIDO, CANCELADO), autor, etapas de origem e destino, comentário, data.
`documento`	Nome original, chave no S3, tipo, tamanho, quem enviou.
`notificacao`	Destinatário, título, mensagem, processo relacionado, lida.

6. Referência da API

Todas as rotas começam com `/api`. Fora as de autenticação, exigem o cabeçalho `Authorization: Bearer <token>`. A documentação interativa fica em `/swagger-ui.html` (25 rotas).

MÉTODO	ROTA	FUNÇÃO	QUEM
POST	`/auth/registrar-empresa`	Cria empresa, administrador e fluxo padrão; devolve sessão	público
POST	`/auth/login`	Autentica e devolve o token	público
GET	`/me`	Dados do usuário logado	todos
GET	`/usuarios`	Lista usuários da empresa	todos
POST	`/usuarios`	Cria usuário	Admin
PATCH	`/usuarios/{id}`	Altera papel ou ativo	Admin
GET	`/fluxos`, `/fluxos/{id}`	Lista e detalha fluxos com etapas e transições	todos
POST	`/fluxos`	Cria fluxo	Admin, Gestor
PATCH	`/fluxos/{id}`	Ativa ou desativa	Admin, Gestor
GET	`/processos`	Lista com filtros: `status`, `etapaId`, `responsavelId`, `busca`, `atrasados`, `meus`, `pagina`, `tamanho`	todos
POST	`/processos`	Cria processo	todos
GET	`/processos/{id}`	Detalhe, próximas etapas possíveis e histórico	todos
POST	`/processos/{id}/avancar`	Move para outra etapa	ver seção 4
POST	`/processos/{id}/comentarios`	Adiciona comentário	todos
PUT	`/processos/{id}/responsavel`	Atribui ou remove responsável	ver seção 4
POST	`/processos/{id}/cancelar`	Cancela com motivo	Admin, Gestor
GET, POST	`/processos/{id}/documentos`	Lista e envia arquivo (multipart, campo `arquivo`)	todos
GET	`/documentos/{id}/download`	Baixa o arquivo	todos
DELETE	`/documentos/{id}`	Remove documento	ver seção 4
GET	`/notificacoes`, `/notificacoes/nao-lidas`	Últimas 50 e contador de não lidas	do próprio usuário
POST	`/notificacoes/{id}/ler`, `/notificacoes/ler-todas`	Marca como lida	do próprio usuário

Erros seguem sempre o formato `{"status": 422, "erro": "mensagem"}`; falhas inesperadas devolvem uma mensagem genérica e o detalhe fica só no log do servidor.

7. Segurança

O que já está implementado

- **Isolamento entre empresas:** o `tenant_id` vem só do token; toda consulta filtra por ele; acesso a recurso de outra empresa dá 404. Há testes automáticos cobrindo processos, comentários, documentos, download e uso de fluxo alheio.
- **Senhas:** guardadas com BCrypt; nunca retornam pela API.
- **Autenticação sem sessão:** JWT assinado (HS256) com validade de 12 horas. O sistema se recusa a iniciar com segredo menor que 32 bytes.
- **Acesso por padrão negado:** só `/api/**` autenticada, login e documentação são liberados; qualquer outra rota é bloqueada.
- **Arquivos:** nome sanitizado, chave única no S3, criptografia no repouso, download intermediado pelo backend com cabeçalho `nosniff`.
- **Validação de entrada:** tamanhos, e-mail, senha mínima e campos obrigatórios validados na API.
- **CORS** restrito às origens configuradas; em produção, com Nginx servindo site e API no mesmo domínio, nem é necessário.

Limitações conhecidas

> Esta lista existe para que nada seja descoberto tarde demais. Itens marcados com **(prioridade)** devem ser resolvidos antes de qualquer uso real.

1. **(prioridade) O token não pode ser revogado.** Desativar um usuário ou mudar seu papel só passa a valer quando o token expira (até 12 horas), porque o filtro não consulta o banco a cada requisição. Quem foi desativado continua com o acesso que tinha nesse período.
2. **(prioridade) Sem limite de tentativas de login**, sem recuperação de senha e sem autenticação em dois fatores.
3. **(prioridade) Segredo padrão de desenvolvimento.** Existe um `JWT_SECRET` padrão para facilitar o uso local; em produção ele *precisa* ser trocado.
4. Upload sem lista de tipos permitidos e sem antivírus (limite de 25 MB por arquivo).
5. O token fica no `localStorage` do navegador. O React escapa o conteúdo, o que reduz o risco, mas uma falha de injeção de script exporia o token. Uma alternativa é cookie `HttpOnly`.
6. O e-mail é único em todo o sistema: uma pessoa não pode ter o mesmo e-mail em duas empresas.
7. O número do processo usa o identificador global, então não é sequencial por empresa (a primeira da empresa pode ser `2026-000137`).
8. Todos da empresa veem todos os processos; não há visibilidade por equipe ou por processo.
9. Não há ferramentas de LGPD (exportar ou excluir dados de uma pessoa), nem registro de auditoria de login.

Limitações funcionais

- Fluxos não podem ser editados depois de criados (só ativados ou desativados); a tela monta apenas transições em sequência.
- Os campos extras do processo são um JSON livre aceito pela API; ainda não há tela para defini-los por fluxo.
- Prazos em dias corridos, sem calendário de feriados.
- Listas de usuários, fluxos e notificações não têm paginação (notificações mostram as 50 mais recentes).
- Data e hora do histórico aparecem no fuso do navegador.

8. Infraestrutura e implantação

Execução local

O `docker-compose.yml` sobe todo o ambiente com um comando (`docker compose up --build`):

SERVIÇO	FUNÇÃO	ENDEREÇO LOCAL
frontend	Nginx servindo o site e repassando `/api`	localhost:3000
backend	API Spring Boot	localhost:8080 (Swagger em `/swagger-ui.html`)
mysql	Banco MySQL 8.4, com volume persistente	interno
s3	S3 simulado (`adobe/s3mock`) só para desenvolvimento	localhost:9090
mailpit	Captura os e-mails enviados	localhost:8025

Configuração por variáveis de ambiente

VARIÁVEL	PARA QUÊ
`JWT_SECRET`	Segredo de assinatura (mínimo 32 bytes). **Obrigatório trocar em produção.**
`DB_URL` , `DB_USER` , `DB_PASSWORD`	Conexão com o MySQL.
`S3_BUCKET` , `S3_REGIAO`	Bucket real. Sem `S3_ENDPOINT` , usa a AWS com credenciais da função (IAM Role).
`MAIL_*`	Servidor SMTP (por exemplo AWS SES) e remetente.
`CORS_ORIGINS`	Origens permitidas.
`PRAZOS_CRON` , `PRAZOS_ANTECEDENCIA_DIAS`	Horário do alerta de prazo e antecedência em dias.

Caminho sugerido para produção na AWS

- **ECS Fargate** para backend e frontend (imagens dos Dockerfiles), atrás de um balanceador com HTTPS.
- **RDS MySQL** com backups automáticos.
- **S3** com bloqueio de acesso público e versionamento; a função do ECS com permissão só nesse bucket.
- **SES** para o envio de e-mails e um domínio verificado.
- Segredos no AWS Secrets Manager, logs no CloudWatch.

A integração contínua (GitHub Actions) compila e testa o backend e compila o frontend a cada envio.

9. Qualidade e verificações

O que foi verificado

VERIFICAÇÃO	RESULTADO
8 testes automáticos de integração (banco H2 em modo MySQL)	**passam**
Ciclo completo do processo: criar, avançar por todas as etapas, concluir, histórico	**coberto por teste**
Isolamento entre empresas (processo, comentário, documentos, download, fluxo alheio)	**coberto por teste**
Permissões por papel e transição fora do fluxo	**coberto por teste**
Upload e download, com nome malicioso sanitizado	**coberto por teste**
Alerta de prazo: dispara uma vez só	**coberto por teste**
Aplicação contra MySQL 8 e S3 simulado reais (teste ponta a ponta via HTTP)	**executado, incluindo acentuação e isolamento**
Interface em navegador automatizado, nos dois temas, e troca de tema com persistência	**executado**
Build do frontend com verificação de tipos	**passa**

Esse teste com MySQL real encontrou dois defeitos que o H2 não revelava (parâmetro de codificação da conexão e health check de e-mail derrubando a aplicação); ambos já foram corrigidos.

O que ainda não foi verificado

- O `docker compose up --build` completo, com as imagens do backend e do frontend rodando em contêiner (as peças foram testadas separadamente).
- AWS S3 e envio de e-mail reais (foram usados simuladores).
- Desempenho e carga, e comportamento com muitas empresas e processos.
- Navegadores além do Chromium, aparelhos móveis e acessibilidade formal.
- Testes automáticos do frontend (hoje só há o build e verificação manual automatizada).

Histórico de versões

COMMIT	MUDANÇA
`91d1473`	Estrutura inicial: backend, frontend, Docker, CI, testes e README
`591d047`	Alternância de tema claro/escuro com botão de lua e sol
`243ed8c`	Refinamento visual profissional
`4469f0f`	Correção: alinhamento dos campos extras no detalhe do processo
`6b83214`	Título do processo também abre o detalhe na lista

10. Próximos passos recomendados

Antes de qualquer uso real (segurança)

1. Validar o usuário a cada requisição (ou usar tokens curtos com renovação) para que desativar e mudar papel tenham efeito imediato.
2. Limite de tentativas de login e recuperação de senha por e-mail.
3. Trocar o segredo JWT e tirar valores padrão do ambiente de produção.
4. Lista de tipos de arquivo permitidos e verificação de vírus nos uploads.

Produto

1. Editor de fluxo com transições livres e edição de fluxos existentes.
2. Campos personalizados por fluxo, com tela de configuração.
3. Painel com indicadores (abertos, atrasados, tempo médio por etapa) e relatórios.
4. Prazos em dias úteis, com calendário de feriados por empresa.
5. Cobrança e planos para vender como SaaS, com limites por empresa.

Engenharia

1. Executar e validar o `docker compose` completo em CI.
2. Testes automáticos do frontend e testes de ponta a ponta.
3. Paginação nas listagens restantes e numeração de processo por empresa.

Apêndice — como acessar

Não há usuário ou senha pré-definidos. Depois de subir o sistema, abra a aplicação, clique em **Cadastrar minha empresa** e crie o seu acesso; ele já nasce como Administrador. Os usuários e as empresas mostradas nas capturas deste relatório foram dados de demonstração criados num ambiente temporário e não fazem parte do código.